

[À COMPLÉTER : nom de l'établissement]

[À COMPLÉTER : filière et spécialité]

RAPPORT DE PROJET DE FIN D'ÉTUDES

Walmart Streaming Flow

Conception et mise en oeuvre d'une plateforme Data Engineering incrémentale pour l'analyse retail

PostgreSQL → Databricks Lakehouse → dbt → Power BI
Génération continue d'événements, traitement horaire et modèle décisionnel en étoile

Réalisé par :	Amine Ait Ali	
Type de projet :	[À COMPLÉTER : PFE, stage ou projet académique à confirmer]	
Organisme d'accueil :	[À COMPLÉTER : nom de l'entreprise ou du laboratoire]	
Encadrant académique :	[À COMPLÉTER : nom et fonction]	Projet
Encadrant professionnel :	[À COMPLÉTER : nom et fonction, si applicable]	
Période :	[À COMPLÉTER : dates de début et de fin]	
Année universitaire :	[À COMPLÉTER : année universitaire]	

pédagogique et de portfolio, sans affiliation à Walmart Inc.

Remerciements

Nous tenons à exprimer notre gratitude à [À COMPLÉTER : nom de l'encadrant académique] pour son accompagnement, ses orientations méthodologiques et la qualité de ses retours durant la réalisation de ce projet. Nous remercions également [À COMPLÉTER : nom de l'encadrant professionnel ou de la structure d'accueil] pour [À COMPLÉTER : nature de l'accompagnement et ressources mises à disposition].

Nous adressons nos remerciements aux enseignants de [À COMPLÉTER : nom de la formation] qui nous ont transmis les fondements nécessaires en bases de données, ingénierie des données, systèmes distribués et informatique décisionnelle. Enfin, nous remercions nos proches pour leur soutien constant. Les choix techniques, analyses et éventuelles erreurs présentés dans ce document restent de notre responsabilité.

Résumé

Ce rapport présente la conception et la réalisation de *Walmart Streaming Flow*, une plateforme pédagogique de Data Engineering destinée à simuler, intégrer, transformer et valoriser des événements retail. Un générateur web en Python et Flask produit deux familles d'événements cohérents avec le modèle métier : l'inscription automatique de nouveaux clients et la création de commandes comportant une à cinq lignes. Les écritures sont réalisées de manière transactionnelle dans une base PostgreSQL externe, tout en préservant les clés étrangères reliant clients, magasins, employés et produits.

Le traitement analytique demeure volontairement micro-batch et horaire. Apache Airflow orchestre onze étapes déterministes : ingestion incrémentale vers Bronze, contrôles de fraîcheur, transformations dbt vers Silver technique puis Silver métier, snapshots historisés et construction de la couche Gold. Le lakehouse Databricks utilise des tables Delta, tandis que dbt formalise les dépendances, la qualité et le modèle dimensionnel. La table de faits est définie au grain d'une ligne de commande et alimente un schéma en étoile consommable dans Power BI.

L'étude détaille les exigences fonctionnelles et non fonctionnelles, les arbitrages d'architecture, les mécanismes d'idempotence et les tests. Une validation de bout en bout confirme la propagation d'un client, d'une commande et de ses lignes jusqu'aux dimensions et à la table de faits. Le résultat constitue une chaîne reproductible et observable, adaptée à la démonstration de changements quasi continus sans remettre en cause un pipeline batch déjà stable.

Mots-clés : Data Engineering, PostgreSQL, Databricks, Delta Lake, dbt, Apache Airflow, Power BI, traitement incrémental, architecture médaillon, schéma en étoile.

Abstract

This report presents the design and implementation of *Walmart Streaming Flow*, an educational data engineering platform built to simulate, ingest, transform and analyze retail events. A Python and Flask web generator produces two business-consistent event families : automatically generated customer sign-ups and orders containing one to five line items. Writes are committed transactionally to an external PostgreSQL database while preserving foreign-key relationships between customers, stores, employees and products.

The analytical processing intentionally remains an hourly micro-batch workflow. Apache Airflow orchestrates eleven deterministic steps : incremental ingestion into Bronze, freshness checks, dbt transformations into technical and business Silver layers, historical snapshots and construction of the Gold layer. Databricks stores the lakehouse data in Delta tables, while dbt formalizes dependencies, data quality rules and dimensional modeling. The fact table is defined at order-line grain and feeds a Power BI-ready star schema.

This study describes functional and non-functional requirements, architectural trade-offs, idempotency mechanisms and verification strategy. An end-to-end validation confirms that a newly created customer, order and line item are propagated into the appropriate dimensions and fact table. The resulting solution is reproducible and observable, and demonstrates continuously changing source data without destabilizing an already reliable batch transformation pipeline.

Keywords : Data Engineering, PostgreSQL, Databricks, Delta Lake, dbt, Apache Airflow, Power BI, incremental processing, medallion architecture, star schema.

Table des matières

Remerciements	i
Résumé	ii
Abstract	iii
Liste des acronymes	viii
Introduction générale	1
1 Contexte général et étude de l'existant	3
1.1 Cadre du projet	3
1.2 État initial et évolution du besoin	3
1.3 Panorama des technologies	3
1.3.1 PostgreSQL et service Ghost	4
1.3.2 Python et Flask	4
1.3.3 Databricks et Delta Lake	4
1.3.4 dbt	4
1.3.5 Apache Airflow et Docker	4
1.3.6 Power BI	5
1.4 Analyse critique de l'existant	5
1.5 Positionnement de la solution	6
2 Analyse et spécification des besoins	7
2.1 Acteurs et périmètre	7
2.2 Besoins fonctionnels	7
2.2.1 Génération d'un client	7
2.2.2 Génération d'une commande	8
2.2.3 Contrôle du générateur	8
2.2.4 Propagation analytique	8
2.3 Besoins non fonctionnels	8
2.4 Contraintes et hypothèses	8
2.5 Règles métier consolidées	8
2.6 Critères d'acceptation	10
3 Conception de la solution	11
3.1 Architecture globale	11
3.2 Modèle relationnel de la source	11
3.3 Transaction de commande	12
3.4 Couches Silver et Gold	12
3.5 Orchestration	13

3.6	Modèle Power BI	13
3.7	Sécurité et résilience	14
4	Réalisation et mise en oeuvre	15
4.1	Organisation du projet	15
4.2	Générateur web	15
4.3	Marqueur de changement PostgreSQL	16
4.4	Modèles dbt incrémentaux	16
4.5	Snapshots et table de faits	17
4.6	Orchestration Docker	17
4.7	Intégration Power BI	18
5	Tests, validation et résultats	19
5.1	Stratégie	19
5.2	Contrôles source	19
5.3	Matrice de tests dbt	20
5.4	Test de bout en bout observé	20
5.5	Résultats et incident analysé	20
5.6	Limites	21
	Conclusion générale et perspectives	22
A	Guide d'exécution et d'exploitation	23
A.1	Prérequis et variables	23
A.2	Commandes principales	23
B	Règles du modèle Power BI	25
B.1	Relations	25
B.2	Mesures de départ	25
C	Matrice de traçabilité	26
C.1	Checklist de démonstration	26
	Bibliographie et ressources techniques	27
	Informations manquantes à me fournir	28

Table des figures

2.1	Cas d'utilisation majeurs	7
3.1	Architecture fonctionnelle de bout en bout	11
3.2	Relations métier de PostgreSQL	12
3.3	Séquence transactionnelle d'une commande	12
3.4	Schéma en étoile de la couche Gold	13
3.5	Ordre des onze tâches Airflow	13
5.1	Chaîne de validation d'un événement	19

Liste des tableaux

1.1	Rôle des technologies dans la solution	5
2.1	Exigences non fonctionnelles	9
4.1	Pages Power BI et questions métier	18
5.1	Principales familles de tests	20
A.1	Variables d’environnement attendues	23
C.1	Traçabilité besoin–implémentation–test	26

Liste des acronymes

ACID	Atomicité, cohérence, isolation et durabilité.
API	Application Programming Interface.
BI	Business Intelligence.
CDC	Change Data Capture ; dans ce projet, détection logique des ajouts et mises à jour par version de changement.
DAG	Directed Acyclic Graph, graphe orienté acyclique.
DAX	Data Analysis Expressions, langage de mesures de Power BI.
DBT/dbt	Data Build Tool, outil de transformation analytique SQL.
ELT	Extract, Load, Transform.
FK	Foreign Key, clé étrangère.
KPI	Key Performance Indicator.
OBT	One Big Table, table métier dénormalisée intermédiaire.
OLTP	Online Transaction Processing.
PFE	Projet de fin d'études.
PK	Primary Key, clé primaire.
SCD	Slowly Changing Dimension.
SQL	Structured Query Language.
SLA	Service Level Agreement.
UI/UX	User Interface / User Experience.

Introduction générale

Contexte et motivation

Les systèmes de vente au détail produisent des données à des rythmes différents : inscriptions de clients, commandes, lignes de commande, mises à jour d'états, informations de produits et organisation des magasins. Ces données prennent une valeur décisionnelle lorsqu'elles sont fiables, historisées et structurées pour répondre rapidement à des questions telles que : quel est le chiffre d'affaires par période ? quels produits contribuent le plus aux ventes ? quelle est la valeur moyenne d'une commande ? comment évolue la performance des magasins ?

Une difficulté récurrente réside dans le passage d'une base opérationnelle mutable vers un modèle analytique stable. Une simple copie complète est facile à comprendre, mais coûteuse, peu réactive et fragile à mesure que le volume croît. À l'inverse, un système intégralement événementiel ou temps réel peut accroître fortement la complexité d'exploitation. Le présent projet étudie une position intermédiaire : la source change continuellement, tandis que le traitement analytique reste incrémental, contrôlé et exécuté toutes les heures.

Walmart Streaming Flow est un projet pédagogique et de portfolio, sans affiliation à Walmart Inc. Il reproduit un domaine retail cohérent à l'aide de données synthétiques. Le système est conçu comme une chaîne complète : génération d'événements, persistance OLTP dans PostgreSQL, ingestion dans Databricks, transformations dbt selon une architecture médaillon, orchestration Apache Airflow et consommation dans Power BI.

Problématique

La problématique centrale peut être formulée ainsi :

Comment injecter de nouveaux événements métier cohérents dans une base PostgreSQL, puis les propager de manière incrémentale et vérifiable jusqu'à un schéma en étoile Power BI, tout en conservant un pipeline horaire existant et sans rompre son modèle métier ?

Cette question implique plusieurs sous-problèmes. Le générateur doit éviter les commandes orphelines et garantir qu'un employé est actif dans le magasin de la commande. L'ingestion doit distinguer les lignes nouvelles ou modifiées de celles déjà traitées. Les transformations doivent séparer données brutes, données conformées et données décisionnelles. Enfin, la couche Gold doit conserver un grain explicite afin que les agrégations Power BI ne multiplient pas artificiellement les montants.

Objectifs

L'objectif général consiste à réaliser une plateforme démontrant un flux de données retail de bout en bout, reproductible sur un poste de développement et exploitable dans un tableau de bord. Les objectifs spécifiques sont les suivants :

- générer automatiquement un nouveau client ou une commande complète depuis une interface web ;
- respecter les relations entre clients, employés, magasins, produits, commandes et lignes ;
- sécuriser l'écriture d'une commande par une transaction atomique ;
- détecter les ajouts et modifications à l'aide d'une version de changement ;
- organiser le lakehouse en couches Bronze, Silver et Gold ;
- historiser les dimensions mutables par des snapshots dbt de type SCD 2 ;
- construire une table de faits au grain de la ligne de commande ;
- automatiser l'ordre d'exécution et les contrôles avec Airflow ;
- fournir un modèle Power BI lisible, composé de relations un-à-plusieurs ;
- documenter les limites et les évolutions envisageables vers une architecture plus temps réel.

Démarche adoptée

Nous avons suivi une démarche itérative et incrémentale. Une première version fonctionnelle du pipeline horaire a servi de référence stable. Le générateur a ensuite été ajouté uniquement au niveau de la source. Chaque évolution a été vérifiée verticalement : validité dans PostgreSQL, présence en Bronze, conformité en Silver, disponibilité en Gold et cohérence du modèle Power BI. Cette façon de procéder réduit le risque de masquer une anomalie derrière plusieurs couches.

Les décisions sont guidées par quatre principes : préserver l'existant fonctionnel, préférer les contrats explicites, rendre les traitements rejouables et tester au même grain que les règles métier. La documentation officielle des technologies complète l'observation du dépôt. Les recommandations Databricks décrivent notamment une progression Bronze–Silver–Gold où la qualité s'accroît couche après couche [1]. dbt formalise le traitement des enregistrements nouveaux ou modifiés dans ses modèles incrémentaux [2].

Organisation du rapport

Le premier chapitre présente le contexte, l'existant et les technologies. Le deuxième formalise les besoins et les contraintes. Le troisième décrit la conception des données, des traitements et du modèle décisionnel. Le quatrième expose la réalisation concrète. Le cinquième détaille les tests, les résultats et les limites. La conclusion synthétise les apports et propose des perspectives. Des annexes fournissent les commandes de démarrage, les règles de calcul Power BI et une matrice de contrôle.

Contexte général et étude de l'existant

1.1 Cadre du projet

Le projet est développé par Amine Ait Ali dans un cadre actuellement identifié comme pédagogique et de portfolio. Les informations relatives à l'établissement, à la structure d'accueil et à la durée officielle n'apparaissent pas dans le dépôt ; elles sont donc signalées dans la section finale des informations manquantes. Cette précaution évite d'attribuer au projet un contexte professionnel non démontré.

Le domaine simulé est celui d'une chaîne de magasins. La plateforme ne cherche pas à reproduire un système de production Walmart réel. Elle fournit un environnement contrôlé permettant de travailler des compétences de Data Engineering : modélisation relationnelle, génération de données, ingestion incrémentale, lakehouse, tests dbt, orchestration et Business Intelligence.

1.2 État initial et évolution du besoin

L'existant comprenait déjà un pipeline stable capable d'extraire les changements d'une base PostgreSQL et de construire des tables analytiques dans Databricks. Le besoin n'était donc pas de réécrire la chaîne, mais de rendre la source vivante. Le générateur devait créer peu d'événements, initialement une commande par minute puis finalement une commande complète toutes les dix secondes, tout en laissant le traitement Airflow à fréquence horaire.

Cette séparation des rythmes est structurante. Le débit source montre une activité continue à l'utilisateur. Le micro-batch horaire concentre les calculs et limite les coûts de traitement. Une ligne créée à l'instant t reste visible dans PostgreSQL immédiatement, puis devient visible en Gold après le prochain passage réussi du DAG et le rafraîchissement Power BI. Le système est donc *streaming côté génération*, mais *micro-batch côté analytique*. L'expression *streaming flow* décrit ici la continuité des événements, pas un engagement de latence sub-seconde de bout en bout.

1.3 Panorama des technologies

1.3.1 PostgreSQL et service Ghost

La base PostgreSQL externe est fournie par un service managé Ghost. Elle constitue la source opérationnelle de vérité. Les six tables du schéma **raw** sont **customers**, **employees**, **stores**, **products**, **orders** et **order_items**. PostgreSQL offre transactions, contraintes, séquences, déclencheurs et verrous consultatifs. Ces derniers conviennent à la sérialisation de courtes sections critiques applicatives sans verrouiller arbitrairement des tables entières [3].

Les secrets de connexion sont injectés par variables d'environnement et ne doivent jamais être copiés dans le code, les captures ou le rapport. La clé d'API et la chaîne PostgreSQL reçues durant le développement sont volontairement absentes de ce document.

1.3.2 Python et Flask

Python implémente la logique de génération. Flask expose une interface locale *Order Pulse*, accessible sur le premier port libre compris entre 5050 et 5099. La page permet de démarrer ou d'arrêter une production continue et de déclencher individuellement les événements. La génération des champs clients est automatique : l'utilisateur n'a pas à saisir manuellement une identité à chaque inscription.

1.3.3 Databricks et Delta Lake

Databricks fournit le moteur de calcul et l'espace lakehouse. Les tables Delta combinent un stockage analytique avec des opérations transactionnelles, un schéma contrôlé et des mises à jour de type **MERGE**. Dans le projet, le catalogue logique s'appelle **walmart**, avec des schémas correspondant aux couches Bronze, Silver et Gold. La documentation Databricks présente le lakehouse comme l'association de propriétés de lac de données et d'entrepôt, notamment pour partager une source analytique cohérente [4].

1.3.4 dbt

dbt transforme les sources Bronze par du SQL versionné. Les dépendances sont exprimées à l'aide de **source()** et **ref()**, ce qui forme un graphe de lignage. Le dépôt contient treize modèles, cinq snapshots et cent sept tests de données au moment de l'analyse. Les modèles Silver techniques sont incrémentaux ; les tables de dimensions utilisent des snapshots horodatés afin de conserver les versions historiques. dbt décrit les snapshots comme une mise en oeuvre des dimensions à variation lente de type 2 [5].

1.3.5 Apache Airflow et Docker

Airflow orchestre le pipeline dans Docker Compose. La configuration utilise Airflow 3.2.2 avec CeleryExecutor, Redis comme courtier, PostgreSQL pour les métadonnées et des composants distincts pour l'API, l'ordonnanceur, le processeur de DAG et le worker. Un DAG représente un workflow dont les tâches et dépendances imposent l'ordre d'exécution [6]. L'ordonnanceur déclenche les tâches prêtes lorsque leurs dépendances sont satisfaites [7].

1.3.6 Power BI

Power BI consomme les tables Gold. Son modèle s'appuie sur une table de faits centrale et cinq dimensions. Microsoft recommande le schéma en étoile pour séparer les tables utilisées pour filtrer et regrouper des tables utilisées pour agréger [8]. Cette règle rend les mesures plus prévisibles et limite les ambiguïtés de filtrage.

TABLE 1.1 – Rôle des technologies dans la solution

Technologie	Rôle principal	Élément vérifiable dans le dépôt
PostgreSQL	Source OLTP et intégrité transactionnelle	Schéma raw , triggers, séquences et clés
Flask/Python	Génération d'événements et interface locale	Application <i>Order Pulse</i>
Databricks	Calcul et stockage Delta	Job distant et catalogue walmart
dbt	Transformation, snapshots, tests et lignage	13 modèles, 5 snapshots, 107 tests
Airflow	Orchestration horaire	DAG orchestrate de 11 tâches
Docker	Reproductibilité locale de l'orchestrateur	Image Airflow et fichier Compose
Power BI	Modèle sémantique et visualisation	Relations étoile et mesures DAX

1.4 Analyse critique de l'existant

L'existant présente plusieurs points forts. Les responsabilités sont séparées : le générateur écrit uniquement dans la source ; Airflow orchestre ; Databricks calcule ; dbt transforme et teste ; Power BI visualise. Le pipeline horaire n'est pas couplé aux clics de l'interface. Les couches permettent d'isoler les erreurs et de rejouer une transformation sans régénérer les événements.

Les difficultés observées concernent surtout le grain et l'historique. Les anciennes commandes ne possédaient pas toujours d'employé, alors que les nouvelles respectaient cette relation. Une correction a affecté aux commandes historiques un employé actif du même magasin, ce qui a permis de relier **dim_employees** à **fact_orders**. Une autre anomalie a révélé que **dim_orders.order_item_id** restait nul pour les commandes générées. La correction n'a pas ajouté une colonne plurielle ; elle a rétabli l'unique colonne **order_item_id** et choisi, pour une dimension au grain commande, la première ligne selon l'identifiant minimal. Ces épisodes montrent pourquoi le grain doit être documenté et testé explicitement.

1.5 Positionnement de la solution

Trois options d'évolution pouvaient être considérées : réécriture vers un bus d'événements temps réel, simulation par fichiers, ou ajout d'un producteur directement connecté à la source existante. La troisième a été retenue. Elle minimise l'impact sur un pipeline opérationnel et offre une démonstration compréhensible. Le compromis est une latence analytique pouvant approcher une heure, à laquelle s'ajoute le délai de rafraîchissement Power BI.

La solution ne constitue donc pas encore une plateforme de streaming analytique stricte. Elle constitue un système hybride pertinent pour un PFE : événements continus en entrée, traitement incrémental et périodique en aval. Une évolution future vers Kafka ou Redpanda pourrait conserver les contrats métier tout en remplaçant progressivement l'ingestion.

Analyse et spécification des besoins

2.1 Acteurs et périmètre

Le système distingue quatre acteurs logiques. L'opérateur du générateur déclenche les événements et surveille l'interface. L'ingénieur Data administre l'orchestration, les transformations et les tests. L'analyste BI construit les mesures et les pages du rapport. Le décideur consulte les indicateurs. Un même utilisateur peut cumuler ces rôles dans le contexte pédagogique.

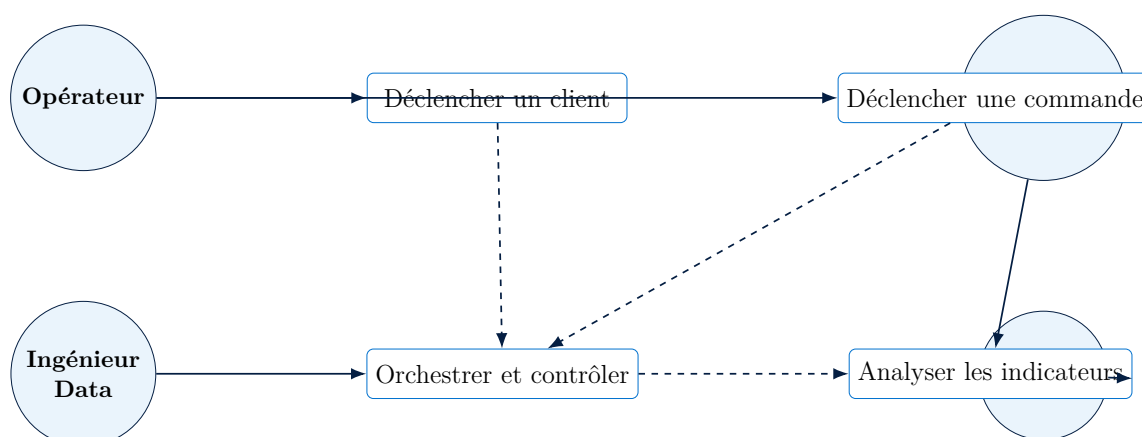


FIGURE 2.1 – Cas d'utilisation majeurs

2.2 Besoins fonctionnels

2.2.1 Génération d'un client

L'interface doit permettre de déclencher un événement d'inscription sans formulaire détaillé. Le service produit automatiquement un identifiant unique, un prénom, un nom, une adresse électronique, un téléphone, une ville et un pays cohérents avec le jeu de données. Une fois persisté, ce client devient éligible à la sélection aléatoire lors des commandes suivantes. Cette règle montre que deux types d'événements peuvent interagir sans saisie manuelle.

2.2.2 Génération d'une commande

Une commande doit référencer un client actif existant, un magasin actif et un employé actif appartenant à ce même magasin. Elle contient de une à cinq lignes, chacune portant un produit actif, une quantité comprise entre un et cinq, un prix unitaire et un montant de ligne calculé. Le total de commande est la somme des lignes. Le paiement et l'état proviennent de domaines autorisés.

La diversité est recherchée sans sacrifier la cohérence. Le client, le magasin, l'employé et les produits varient. Lorsque plusieurs magasins sont disponibles, l'algorithme évite autant que possible de sélectionner immédiatement le même magasin. De même, il tente de ne pas répéter le dernier client. Il ne s'agit pas d'une garantie statistique d'uniformité, mais d'une amélioration de la variété visible.

2.2.3 Contrôle du générateur

L'utilisateur doit pouvoir démarrer et arrêter la boucle, régler l'intervalle entre deux commandes dans les limites prévues, lancer un événement unitaire et consulter un journal succinct. Le port d'écoute doit éviter les collisions locales en recherchant un port libre dans la plage 5050–5099. Le serveur reste lié à l'interface locale pour réduire l'exposition accidentelle.

2.2.4 Propagation analytique

Les changements nouveaux ou modifiés doivent être chargés en Bronze, conformés en Silver et publiés en Gold. L'ordre des étapes doit être automatique et l'échec d'un contrôle doit empêcher une publication incohérente. Power BI doit pouvoir importer ou interroger les dimensions actuelles et la table de faits.

2.3 Besoins non fonctionnels

2.4 Contraintes et hypothèses

Le service PostgreSQL, le workspace Databricks et Power BI sont externes au réseau Docker local. Leur disponibilité dépend du réseau et des quotas associés. Les identifiants techniques doivent être fournis à l'exécution. L'orchestration Airflow s'exécute dans des conteneurs Linux, tandis que le générateur est lancé sur l'hôte Windows.

Les données sont synthétiques. Elles permettent de valider la logique, mais ne garantissent pas une distribution réaliste de la demande, des saisons, des promotions ou des comportements clients. Les dates historiques du jeu initial et les événements générés doivent être interprétés comme un laboratoire et non comme des ventes réelles.

2.5 Règles métier consolidées

1. Un client peut passer plusieurs commandes ; une commande appartient à un seul client.

TABLE 2.1 – Exigences non fonctionnelles

Qualité	Exigence	Mécanisme retenu
Intégrité	Aucune commande ne doit référencer une entité inexistante	Sélection de clés actives, FK et transaction
Atomicité	Une commande et ses lignes sont validées ensemble	Une transaction avec annulation en cas d'erreur
Idempotence	Un rejeu ne doit pas multiplier les lignes	Clé unique et stratégie incrémentale merge
Traçabilité	Les changements et périodes de validité sont observables	change_version , timestamps et snapshots
Testabilité	Les règles critiques doivent être exécutables	Tests dbt génériques et personnalisés
Sécurité	Aucun secret ne doit être commité ou exposé	Variables d'environnement et exemples anonymisés
Maintenabilité	Le pipeline existant doit rester stable	Ajout limité à la source et contrats de colonnes
Utilisabilité	Le générateur doit être utilisable sans SQL	Interface Flask simple et actions explicites
Performance	Éviter les recalculs complets réguliers	Filtres incrémentaux et traitement horaire

2. Un magasin emploie plusieurs employés ; l'employé choisi pour une commande doit appartenir à ce magasin.
3. Une commande possède au moins une ligne ; une ligne appartient à une seule commande.
4. Une ligne référence un seul produit et contient une quantité strictement positive.
5. Le montant de ligne vaut $q \times p$, où q est la quantité et p le prix unitaire, sous réserve des règles d'arrondi.
6. Le total de commande vaut la somme de ses lignes.
7. Une dimension historisée peut contenir plusieurs versions techniques d'un même identifiant métier, mais une seule version courante.
8. La table de faits contient une ligne par `order_item_id` ; le montant de vente agrégable est `sales_amount`.
9. `order_total_amount` est répété sur les lignes d'une commande et ne doit jamais être additionné directement dans Power BI.
10. Dans `dim_orders`, `order_item_id` désigne la première ligne de la commande, choisie de manière déterministe ; il ne change pas le grain commande de la dimension.

2.6 Critères d'acceptation

Le système est accepté si un client créé apparaît dans la dimension client après exécution, si une commande et toutes ses lignes apparaissent dans la table de faits, si les clés de dimensions trouvent une correspondance, si aucun `order_item_id` généré n'est nul dans les tables attendues, et si les tests dbt ciblés réussissent. Le rapport Power BI doit ensuite afficher une variation mesurable après rafraîchissement, par exemple une augmentation du nombre de commandes et du chiffre d'affaires.

Conception de la solution

3.1 Architecture globale

La solution sépare clairement le plan opérationnel, le plan de traitement et le plan de consommation. Le générateur Flask écrit exclusivement dans PostgreSQL. Airflow pilote un job Databricks et les commandes dbt. Les tables Delta organisent les données selon trois niveaux de confiance, puis Power BI consomme uniquement la couche Gold. Cette séparation empêche un clic dans l'interface de devenir dépendant d'un traitement analytique long.

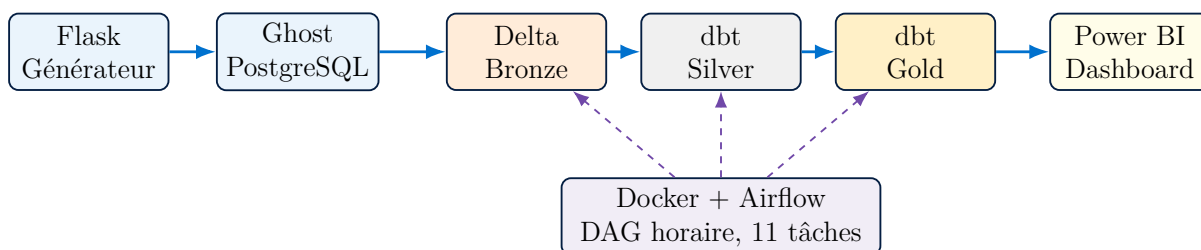


FIGURE 3.1 – Architecture fonctionnelle de bout en bout

La génération est continue, alors que le calcul reste micro-batch. Databricks recommande une progression Bronze–Silver–Gold dans laquelle la qualité et la valeur métier augmentent à chaque couche [1]. Ce modèle répond au besoin de préserver un pipeline horaire stable tout en rendant la source dynamique.

3.2 Modèle relationnel de la source

La base source contient six entités. Un client passe plusieurs commandes. Un magasin possède plusieurs employés et reçoit plusieurs commandes. Une commande est traitée par un employé du même magasin. Une commande contient une à cinq lignes, et chaque ligne référence un produit.

Les tables `orders` et `order_items` portent un `change_version` alimenté par une séquence et un trigger. La valeur croissante sert de curseur d'extraction. Elle évite les ambiguïtés d'un timestamp seul et permet de récupérer les insertions comme les mises à jour.

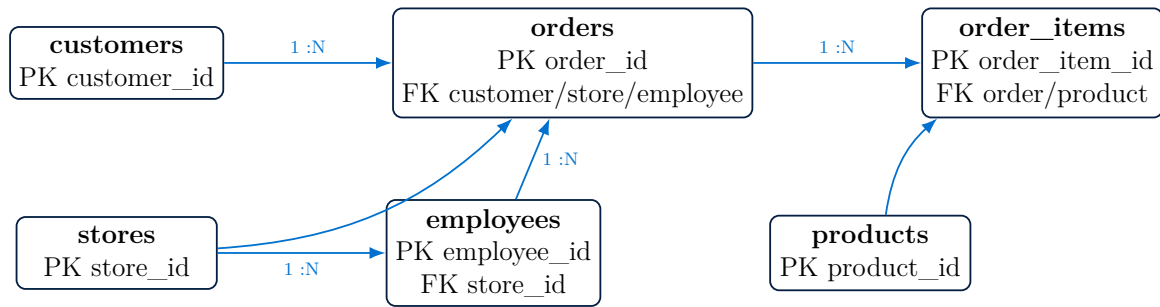


FIGURE 3.2 – Relations métier de PostgreSQL

3.3 Transaction de commande

Une commande et ses lignes forment un agrégat atomique. Le générateur ouvre une transaction, acquiert un verrou consultatif, sélectionne des références actives, calcule les montants, insère l'en-tête et les lignes, puis valide. Une exception provoque un rollback complet. PostgreSQL fournit des verrous consultatifs adaptés aux sections critiques applicatives [3].

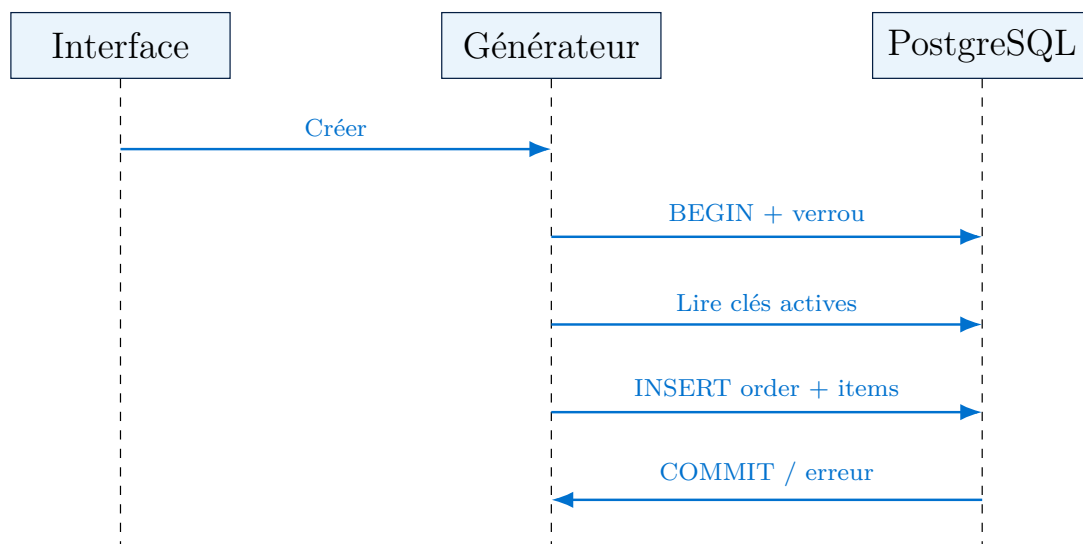


FIGURE 3.3 – Séquence transactionnelle d'une commande

La variabilité reste contrôlée. Le client est tiré parmi les clients actifs; le magasin parmi les magasins actifs; l'employé parmi les employés actifs de ce magasin. Cette sélection conditionnelle évite l'erreur logique d'un identifiant inexistant ou d'un employé affecté ailleurs. Le service tente aussi de ne pas répéter immédiatement le dernier client et le dernier magasin.

3.4 Couches Silver et Gold

Les six modèles Silver techniques sont incrémentaux, avec une clé unique et une stratégie **merge**. Lors du premier lancement, toutes les lignes sont transformées. Ensuite, `is_incremental()` restreint la lecture aux versions nouvelles. dbt indique qu'une clé unique permet de mettre à jour une ligne existante au lieu de la dupliquer [2].

Silver métier matérialise `obt_b`, une vue dénormalisée au grain ligne de commande. Elle réunit le contexte client, produit, magasin et employé. Gold expose cinq dimensions historisées et une table de faits. Les snapshots dbt implémentent des dimensions à variation lente de type 2 [5].

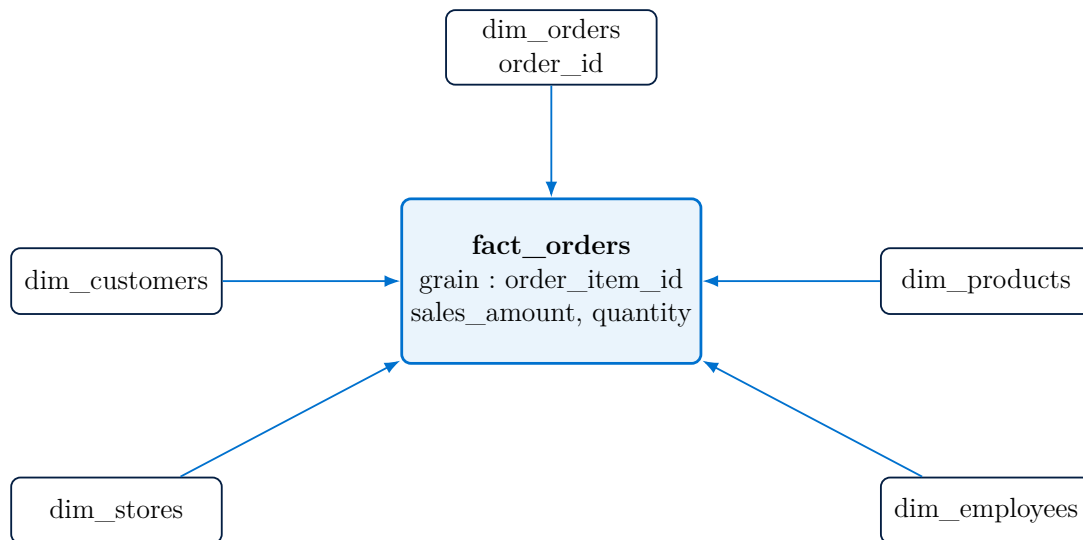


FIGURE 3.4 – Schéma en étoile de la couche Gold

Le fait possède une ligne par `order_item_id`. `sales_amount` est additive. `order_total_amount` est répété pour chaque ligne et ne doit pas être additionné dans Power BI. La dimension commande conserve une ligne par `order_id`. Sa colonne `order_item_id` reçoit de manière déterministe l'identifiant minimal des lignes de la commande. Elle sert à la traçabilité ; l'analyse détaillée utilise le fait.

3.5 Orchestration

Le DAG `orchestrate`, planifié avec `@hourly` et `catchup=False`, exécute onze tâches. Les tests placés entre les couches sont des portes de qualité : une défaillance bloque la publication aval. Airflow représente le workflow comme un DAG et exécute les tâches lorsque leurs dépendances sont satisfaites [6, 7].

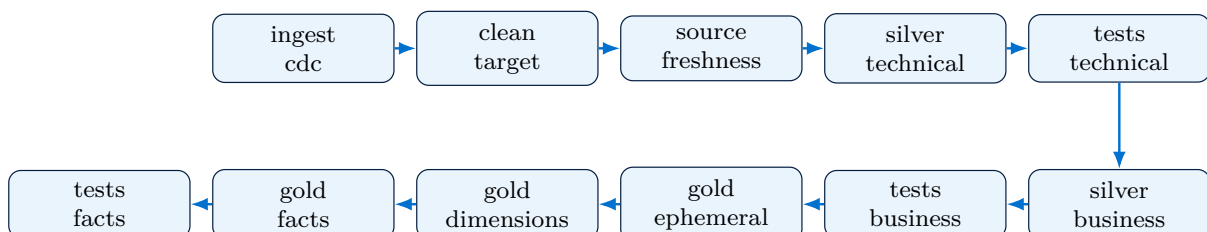


FIGURE 3.5 – Ordre des onze tâches Airflow

3.6 Modèle Power BI

Power BI importe les versions courantes des dimensions et `fact_orders`. Les relations sont actives, de cardinalité 1 :N, et filtrent de la dimension vers le fait. Cette conception

correspond aux recommandations Microsoft : les dimensions filtrent et regroupent, tandis que le fait porte les observations numériques [8].

La table de sauvegarde `dim_orders_backup_pre_grain_fix_20260730` est exclue du modèle. Elle n'est pas une dimension fonctionnelle et introduirait des relations ambiguës. La relation employés–fait est désormais valide grâce au traitement de l'historique. Une éventuelle relation magasin–employé ne doit pas créer un second chemin actif vers le fait.

3.7 Sécurité et résilience

Les secrets sont injectés par l'environnement et ne sont jamais reproduits dans le dépôt. L'interface écoute sur `127.0.0.1`. En production, il faudrait ajouter un coffre de secrets, des comptes de service au moindre privilège et une authentification de l'interface.

La résilience repose sur trois mécanismes : rollback à la source, incréments rejouables et tâches Airflow indépendantes. Les journaux du générateur, d'Airflow, de dbt et de Databricks forment une chaîne d'observation. Une évolution future pourra centraliser les métriques de volumes, latence et erreurs.

Réalisation et mise en oeuvre

4.1 Organisation du projet

Le dépôt sépare l'orchestration dans `airflow`, les transformations dans `airflow/walmart_dbt`, le générateur dans le dossier du jeu Walmart et la documentation dans `Docs`. Cette séparation rend les responsabilités visibles et facilite les changements ciblés.

```

1 Walmart Streaming Flow/
2 |-- airflow/
3 | |-- dags/orchestration.py
4 | |-- docker-compose.yaml
5 | '-- walmart_dbt/{models,snapshots,tests}
6 |-- Walmart Data Engineering/Walmart dataset/
7 | |-- data/
8 | '-- templates/generator_index.html
9 |-- Docs/
10 '-- README.md

```

Listing 4.1 – Arborescence fonctionnelle simplifiée

4.2 Générateur web

Au démarrage, l'application charge les variables de connexion, vérifie PostgreSQL et recherche le premier port libre entre 5050 et 5099. La page *Order Pulse* expose deux événements unitaires et une boucle continue. Le client est généré automatiquement ; l'utilisateur ne remplit pas une fiche à chaque essai.

```

1 def build_order(connection):
2     with connection.transaction():
3         acquire_transaction_lock(connection)
4         customer = choose_active_customer(avoid=last_customer_id)
5         store = choose_active_store(avoid=last_store_id)
6         employee = choose_active_employee(store_id=store.id)
7         products = choose_active_products(random_between(1, 5))
8
9         lines = []
10        for product in products:
11            quantity = random_between(1, 5)
12            amount = round(quantity * product.price, 2)
13            lines.append((product.id, quantity, amount))
14

```



```

15     order_id = insert_order(customer, store, employee, lines)
16     insert_order_items(order_id, lines)
17     return order_id

```

Listing 4.2 – Pseudocode de création cohérente d’une commande

Les requêtes sont paramétrées et le commit intervient après toutes les insertions. Le nouvel inscrit devient éligible lors des commandes suivantes parce que la liste des clients actifs est relue depuis la base. Le générateur ne déclenche ni Airflow ni dbt ; la fréquence analytique reste indépendante.

4.3 Marqueur de changement PostgreSQL

Une séquence et un trigger attribuent une version croissante aux commandes et lignes. Un trigger BEFORE peut modifier la ligne avant son insertion ou sa mise à jour, ce qui convient à ce marqueur [9].

```

1 CREATE SEQUENCE IF NOT EXISTS raw.change_version_seq;
2
3 CREATE OR REPLACE FUNCTION raw.set_change_version()
4 RETURNS trigger AS $$
5 BEGIN
6     NEW.change_version := nextval('raw.change_version_seq');
7     RETURN NEW;
8 END;
9 $$ LANGUAGE plpgsql;
10
11 CREATE TRIGGER orders_change_version
12 BEFORE INSERT OR UPDATE ON raw.orders
13 FOR EACH ROW EXECUTE FUNCTION raw.set_change_version();

```

Listing 4.3 – Principe de change_version

L’ajout de `employee_id` a nécessité une migration de l’historique. Plutôt que de supprimer les anciennes commandes, une règle les a associées à un employé actif du même magasin. Cette correction préserve les ventes historiques et rend possible la relation `dim_employees[employee_id]` vers `fact_orders[employee_id]`.

4.4 Modèles dbt incrémentaux

Le projet utilise dbt-core 1.11.12 et dbt-databricks 1.12.2. Les modèles Silver techniques suivent la structure suivante :

```

1 {{ config(
2     materialized='incremental',
3     unique_key='order_id',
4     incremental_strategy='merge'
5 ) }}
6
7 select order_id, employee_id, change_version, updated_at
8 from {{ source('bronze', 'orders') }}
9 {% if is_incremental() %}
10 where change_version > (
11     select coalesce(max(change_version), 0) from {{ this }}

```

```

12 )
13 {% endif %}

```

Listing 4.4 – Structure conceptuelle d'un modèle Silver

La clé unique empêche le jeu de produire des doublons. Une modification de schéma ou de logique historique peut toutefois nécessiter un `-full-refresh`; dbt précise que ce mode reconstruit la table incrémentale entière [2].

`obt_b` joint ensuite les six entités. Comme une commande possède plusieurs lignes, les colonnes d'en-tête sont répétées. Le modèle Gold doit donc distinguer les champs descriptifs des mesures réellement additives.

4.5 Snapshots et table de faits

Les dimensions clients, employés, commandes, produits et magasins utilisent une stratégie timestamp. Chaque version reçoit `dbt_valid_from` et `dbt_valid_to`. La date 9999-12-31 représente la ligne courante.

```

1 {% snapshot dim_customers %}
2 {{ config(
3     unique_key='customer_id',
4     strategy='timestamp',
5     updated_at='customer_updated_at',
6     dbt_valid_to_current="to_timestamp('9999-12-31')"
7 ) }}
8 select * from {{ ref('eph_customers') }}
9 {% endsnapshot %}

```

Listing 4.5 – Snapshot SCD 2 conceptuel

La table de faits conserve toutes les lignes. Pour la dimension commande, une fenêtre sélectionne une valeur représentative sans créer de colonne plurielle :

```

1 with ranked as (
2     select order_id, order_item_id,
3           row_number() over (
4               partition by order_id order by order_item_id
5           ) as item_rank
6     from {{ ref('obt_b') }}
7 )
8 select order_id, order_item_id
9 from ranked
10 where item_rank = 1

```

Listing 4.6 – Correction de `order_item_id` dans `dim_orders`

Cette correction remplit les commandes générées d'identifiant supérieur à 10 000 et conserve le grain d'une ligne par commande. La colonne erronée `order_item_ids` n'appartient pas au modèle final.

4.6 Orchestration Docker

Docker Compose démarre six services : serveur API Airflow, scheduler, processeur de DAG, worker Celery, Redis et PostgreSQL de métadonnées. L'image Airflow installe

le SDK Databricks et dbt. Le DAG exécute exactement les étapes ingestion, nettoyage, fraîcheur, Silver technique, tests techniques, Silver métier, tests métier, éphémères Gold, dimensions, faits et tests des faits.

```

1 ingest_cdc >> clean_target >> source_freshness
2 source_freshness >> silver_technical >> silver_technical_tests
3 silver_technical_tests >> silver_business >> silver_business_tests
4 silver_business_tests >> gold_ephemeral >> gold_dimensions
5 gold_dimensions >> gold_facts >> gold_facts_tests

```

Listing 4.7 – Topologie simplifiée du DAG

Un échec bloque les tâches aval. Airflow porte l'état et les dépendances, tandis que le job Databricks réalise l'ingestion et dbt exécute les transformations.

4.7 Intégration Power BI

Power BI charge `fact_orders` et les cinq dimensions courantes. Les relations sont simples et unidirectionnelles. Les mesures de base sont :

```

1 Revenue = SUM ( fact_orders[sales_amount] )
2 Orders = DISTINCTCOUNT ( fact_orders[order_id] )
3 Items = SUM ( fact_orders[quantity] )
4 Average Order Value = DIVIDE ( [Revenue], [Orders], 0 )

```

Listing 4.8 – Mesures DAX fondamentales

La somme de `order_total_amount` est évitée parce que cette valeur est répétée au grain ligne. Les pages proposées sont : vue exécutive, tendances, produits, magasins, clients, commandes et paiements, analyses avancées, employés, puis qualité du pipeline.

TABLE 4.1 – Pages Power BI et questions métier

Page	Question principale
Vue exécutive	Quelle est la performance globale ?
Tendances	Comment ventes et commandes évoluent-elles ?
Produits	Quelles catégories contribuent le plus ?
Magasins	Quelles implantations surperforment ?
Clients	Quels segments sont fréquents et rentables ?
Commandes	Quels statuts, paiements et paniers dominant ?
Employés	Quelle activité est associée aux employés ?
Qualité	Les données sont-elles fraîches et complètes ?

Tests, validation et résultats

5.1 Stratégie

La validation suit l'identité d'un événement à quatre niveaux : transaction PostgreSQL, tables Bronze, modèles Silver et tables Gold. Elle est complétée par le rafraîchissement Power BI. Le dépôt contient 107 tests dbt portant sur la non-nullité, l'unicité, les relations, les timestamps et les calculs métier.

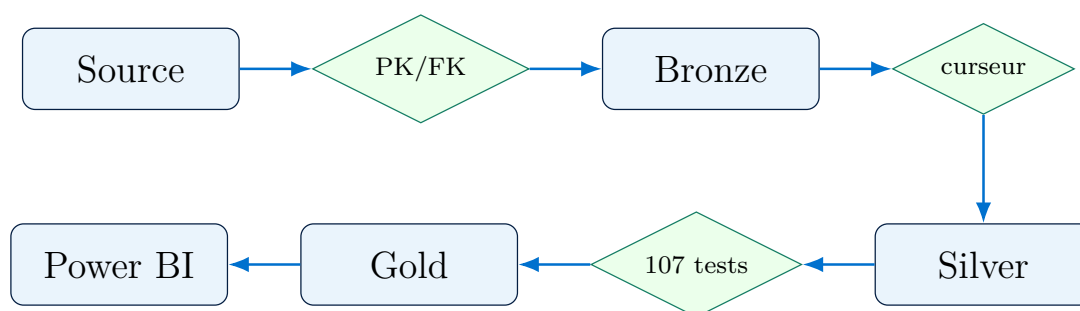


FIGURE 5.1 – Chaîne de validation d'un événement

5.2 Contrôles source

Les requêtes suivantes doivent retourner zéro :

```

1 select count(*)
2 from raw.order_items i
3 left join raw.orders o on o.order_id = i.order_id
4 where o.order_id is null;
5
6 select count(*)
7 from raw.orders o
8 left join raw.employees e on e.employee_id = o.employee_id
9 where e.employee_id is null or e.store_id <> o.store_id;

```

Listing 5.1 – Contrôles d'intégrité PostgreSQL

Un contrôle complémentaire compare le total d'en-tête à la somme des lignes. Un test d'erreur forcée doit montrer qu'aucun en-tête orphelin ne subsiste après rollback.

5.3 Matrice de tests dbt

TABLE 5.1 – Principales familles de tests

Niveau	Exemple	Risque couvert
Silver technique	Clé unique	Doublon après rejeu
Silver métier	Montant de ligne	Incohérence financière
Gold dimension	Une version courante	Doublon côté 1 de Power BI
Gold fait	Item unique et non nul	Grain instable
Intertables	Employé du magasin	Relation métier invalide
Régression	Item minimal par commande	Nullité des nouvelles commandes

Pour les commandes supérieures à 10 000, le test compare la référence de ligne de `dim_orders` au minimum calculé dans `fact_orders`. Il valide la règle exacte, et pas seulement la non-nullité.

5.4 Test de bout en bout observé

Un scénario exécuté durant le développement a suivi le client 2002, la commande 10015 et la ligne 30070. Les enregistrements ont été vus dans PostgreSQL, puis les onze tâches Airflow ont réussi. Les clés ont été retrouvées en Bronze, Silver et Gold, avant leur chargement dans Power BI. Ces identifiants documentent un cas de recette ; la volumétrie courante peut avoir augmenté depuis.

```

1 select order_id, order_item_id
2 from walmart.gold.dim_orders
3 where order_id > 10000
4     and dbt_valid_to = timestamp('9999-12-31');
5
6 select f.order_id, min(f.order_item_id), d.order_item_id
7 from walmart.gold.fact_orders f
8 join walmart.gold.dim_orders d on d.order_id = f.order_id
9 where f.order_id > 10000
10 group by f.order_id, d.order_item_id
11 having min(f.order_item_id) <> d.order_item_id
12     or d.order_item_id is null;
```

Listing 5.2 – Recette Gold des nouvelles commandes

La seconde requête doit retourner zéro ligne. Un rafraîchissement Power BI est ensuite nécessaire : un import local ne se met pas à jour uniquement parce que Gold a changé.

5.5 Résultats et incident analysé

Le pipeline relie désormais les deux événements générés aux dimensions et au fait. L'historique des employés est complet, la relation 1 :N peut être créée, et les commandes

générées possèdent un `order_item_id` conforme. La table de sauvegarde pré-correction est exclue du modèle BI.

Un incident Airflow a interrompu `dbt run -select silver_b` par `SIGTERM` après que l'état de la task instance a été marqué en échec. Aucun message SQL dbt n'était présent avant l'arrêt. Les exécutions ultérieures ayant réussi, l'incident est interprété comme une interruption d'orchestration ou de synchronisation d'état. Le warning de sémaphores est une conséquence probable de l'arrêt brutal, pas une preuve de sa cause.

5.6 Limites

La latence analytique peut approcher une heure. Les données synthétiques ne reproduisent pas toutes les saisons et promotions. Le modèle ne possède pas encore une dimension date Gold dédiée. Les métriques de coût, délai P95 et disponibilité ne sont pas mesurées sur une période assez longue pour annoncer un SLA.

L'environnement Docker local n'est pas hautement disponible et les secrets ne sont pas centralisés. Une page de supervision devrait consolider la dernière version traitée, les volumes par couche, la durée du DAG et les erreurs de tests.

La réussite ne se limite pas à voir un nombre changer dans Power BI : elle consiste à prouver qu'un événement conserve ses clés, son grain et ses règles métier à travers chaque contrat de données.

Conclusion générale et perspectives

Ce projet a permis de construire une chaîne Data Engineering complète depuis une source transactionnelle vivante jusqu'à un modèle décisionnel. Le générateur crée automatiquement des clients et des commandes diversifiées, tout en empêchant les références inexistantes et les incohérences magasin-employé. La transaction garantit qu'une commande ne reste jamais sans ses lignes.

Le pipeline horaire existant a été conservé. Les versions de changement et les modèles dbt incrémentaux limitent le calcul aux données nouvelles ou modifiées. L'architecture Bronze-Silver-Gold sépare fidélité source, conformité métier et consommation. Les snapshots préservent l'historique, tandis que `fact_orders` fixe le grain d'une ligne de commande.

Deux corrections illustrent l'importance du grain. L'historique des commandes a été enrichi avec un employé actif du même magasin plutôt que supprimé. La dimension commande a conservé une seule colonne `order_item_id`, choisie de façon déterministe, au lieu d'ajouter une liste ambiguë. Les tests de régression protègent ces décisions.

Power BI exploite désormais un schéma en étoile : cinq dimensions filtrent le fait, et les mesures s'appuient sur `sales_amount`, le décompte distinct des commandes et la somme des quantités. Le projet mobilise ainsi PostgreSQL, Python/Flask, Databricks, Delta Lake, dbt, Airflow, Docker et Power BI dans un cas cohérent de bout en bout.

À court terme, nous recommandons une dimension date, une page de supervision et une gestion centralisée des secrets. À moyen terme, le générateur pourrait produire changements d'état, retours, annulations et variations de prix. À long terme, Kafka ou Redpanda et une ingestion Databricks streaming pourraient réduire la latence, sous réserve d'un besoin métier mesuré.

En conclusion, *Walmart Streaming Flow* atteint son objectif pédagogique : rendre visible, testable et explicable le parcours d'une donnée depuis son événement opérationnel jusqu'à son usage décisionnel. La valeur du système réside autant dans les indicateurs finaux que dans les garanties placées entre chaque étape.

Guide d'exécution et d'exploitation

A.1 Prérequis et variables

Le poste doit disposer de Docker Desktop, Python 3.11 à 3.13, d'un accès à PostgreSQL Ghost et Databricks, ainsi que de Power BI Desktop. Les secrets sont fournis à l'exécution et ne sont pas reproduits dans ce document.

TABLE A.1 – Variables d'environnement attendues

Variable	Usage
POSTGRES_CONNECTION	Connexion TLS à PostgreSQL
GHOST_API_KEY	Accès Ghost lorsqu'il est requis
DATABRICKS_HOST	Adresse du workspace
DATABRICKS_TOKEN	Jeton d'accès
DATABRICKS_JOB_ID	Job déclenché par Airflow

A.2 Commandes principales

Depuis le dossier `airflow` :

```
1 docker compose build
2 docker compose up -d
3 docker compose ps
```

Listing A.1 – Démarrage Docker

Le générateur est ensuite lancé depuis le dossier du jeu Walmart. Le port normal est `http://127.0.0.1:5050`, avec recherche automatique jusqu'à 5099. Pour une démonstration : créer un client, créer une commande, relever les identifiants, déclencher le DAG si nécessaire, contrôler Gold, puis rafraîchir Power BI.

```
1 dbt debug
2 dbt source freshness
3 dbt run --select silver_t
4 dbt test --select silver_t
5 dbt run --select silver_b
6 dbt test --select silver_b
```



```
7 dbt run --select gold_e
8 dbt snapshot
9 dbt run --select gold_f
10 dbt test --select gold_f
```

Listing A.2 – Commandes dbt de contrôle

Règles du modèle Power BI

B.1 Relations

Les cinq relations vont de la dimension côté 1 vers **fact_orders** côté N : clients par **customer_id**, produits par **product_id**, magasins par **store_id**, employés par **employee_id** et commandes par **order_id**. Le filtre croisé est simple. Les dimensions importées doivent être limitées à leur version courante.

Si Power BI signale un doublon côté 1, il faut vérifier le filtre de snapshot au lieu de forcer une relation plusieurs-à-plusieurs. Cette dernière masquerait un défaut de modèle.

B.2 Mesures de départ

```
1 Revenue = SUM ( fact_orders[sales_amount] )
2 Orders = DISTINCTCOUNT ( fact_orders[order_id] )
3 Order Lines = DISTINCTCOUNT ( fact_orders[order_item_id] )
4 Items = SUM ( fact_orders[quantity] )
5 Average Order Value = DIVIDE ( [Revenue], [Orders], 0 )
6 Average Selling Price = DIVIDE ( [Revenue], [Items], 0 )
```

Listing B.1 – Mesures DAX recommandées

Une page exécutive contient quatre cartes, une tendance et un classement des magasins. Les pages détaillées répondent chacune à un thème. La page employés mesure l'activité enregistrée, mais ne doit pas transformer le revenu associé en jugement automatique de productivité.

Matrice de traçabilité

TABLE C.1 – Traçabilité besoin–implémentation–test

ID	Besoin	Implémentation	Vérification
F01	Inscription automatique	Événement client Flask	Client dans dim
F02	Commande cohérente	Clés actives	Zéro orphelin
F03	Employé du magasin	Filtre par store	Test intertables
F04	Une à cinq lignes	Produits actifs	Comptage par ordre
F05	Atomicité	Transaction	Rollback forcé
F06	Changements	change_version	Curseur Bronze
F07	Rejeu sûr	merge + clé unique	Test unique
F08	Historique	Snapshot timestamp	Version courante
F09	Grain du fait	order_item_id	Unique et non nul
F10	BI	Schéma en étoile	Relations 1 :N
NF01	Secrets	Environnement	Scan du dépôt
NF02	Observabilité	Logs par composant	Inspection des runs

C.1 Checklist de démonstration

1. Vérifier les conteneurs Docker.
2. Créer un client et une commande depuis la page web.
3. Contrôler les relations dans PostgreSQL.
4. Déclencher le DAG ou attendre l'intervalle horaire.
5. Confirmer le succès des onze tâches.
6. Exécuter les requêtes de recette Gold.
7. Rafraîchir Power BI.
8. Contrôler `Orders`, `Revenue` et `Items`.

Bibliographie et ressources techniques

- [1] DATABRICKS. *What is the medallion lakehouse architecture?* URL : <https://docs.databricks.com/aws/en/lakehouse/medallion> (visité le 03/08/2026).
- [2] DBT LABS. *Configure incremental models.* URL : <https://docs.getdbt.com/docs/build/incremental-models> (visité le 03/08/2026).
- [3] POSTGRESQL GLOBAL DEVELOPMENT GROUP. *Explicit Locking – Advisory Locks.* URL : <https://www.postgresql.org/docs/current/explicit-locking.html#ADVISORY-LOCKS> (visité le 03/08/2026).
- [4] DATABRICKS. *What is a data lakehouse?* URL : <https://docs.databricks.com/aws/en/lakehouse> (visité le 03/08/2026).
- [5] DBT LABS. *Add snapshots to your DAG.* URL : <https://docs.getdbt.com/docs/build/snapshots> (visité le 03/08/2026).
- [6] APACHE SOFTWARE FOUNDATION. *Apache Airflow – Architecture Overview.* URL : <https://airflow.apache.org/docs/apache-airflow/stable/core-concepts/overview.html> (visité le 03/08/2026).
- [7] APACHE SOFTWARE FOUNDATION. *Apache Airflow – Scheduler.* URL : <https://airflow.apache.org/docs/apache-airflow/stable/concepts/scheduler.html> (visité le 03/08/2026).
- [8] MICROSOFT. *Understand star schema and the importance for Power BI.* URL : <https://learn.microsoft.com/en-us/power-bi/guidance/star-schema> (visité le 03/08/2026).
- [9] POSTGRESQL GLOBAL DEVELOPMENT GROUP. *Overview of Trigger Behavior.* URL : <https://www.postgresql.org/docs/current/trigger-definition.html> (visité le 03/08/2026).

Informations manquantes à me fournir

Les éléments suivants ne sont pas démontrés par le dépôt et doivent être renseignés avant le dépôt académique définitif :

1. nom officiel de l'établissement et logo autorisé ;
2. filière, spécialité, niveau et diplôme ;
3. nature exacte : PFE, stage ou projet académique ;
4. organisme d'accueil, si applicable ;
5. noms et fonctions des encadrants ;
6. dates, durée et année universitaire ;
7. composition du jury et date de soutenance ;
8. méthodologie exigée par l'établissement ;
9. objectifs chiffrés de fraîcheur, disponibilité et coût ;
10. captures finales autorisées du dashboard Power BI.

Tous les marqueurs [À COMPLÉTER : ...] devront être remplacés. Une dernière compilation et une relecture humaine restent obligatoires avant soutenance.